

Metro Ticket Generation System

Phase 3 — UI/UX Development & Customization | ServiceNow

Phase Purpose	Convert the backend foundation into a usable passenger-facing Service Catalog experience.
Primary Activities	Interface design, catalog item, variables, navigation flow, Catalog Client Scripts, UI Policies and QR-ticket experience.
Main Catalog Item	Book A Metro Ticket
Expected Outcome	A dynamic booking form that guides users, calculates fare values, conditionally displays fields and presents a QR-ticket experience.

1. Phase 3 Overview

Phase 3 focuses on the user experience layer. The goal is to make the Metro Ticket Generation System easy to use while connecting the interface to the backend data and business logic created in Phase 2.

Backend Data → Service Catalog Item → Variables → Dynamic Client Logic → UI Policies → QR Widget → Checkout/Ticket Experience

2. UI/UX Design Principles

- Simple: Ask the passenger only for information required to generate the ticket.
- Guided: Use references, choices and conditional fields so the form explains itself.
- Dynamic: Fare values should react to station, journey type and passenger count.
- Safe: Invalid or incomplete input should be blocked before submission.
- Consistent: Use clear labels such as Starting From, Going To, Type of Journey and No. of Passengers.
- Responsive: The catalog experience should work cleanly in the intended ServiceNow interface.

3. Navigation Flow

Application Navigator → Service Catalog → Maintain Items → Book A Metro Ticket → Variables / Client Scripts / UI Policies → Test Catalog Form

4. Phase 3 Brainstorming

- Which fields should be visible immediately and which should appear only when needed?
- Should Starting From and Going To reference metro station records?
- Should fare fields be read-only so users cannot manually alter calculated amounts?
- What happens when passenger count changes after route selection?
- How should Single Journey and Return Journey affect displayed amount?
- When should the QR code be generated—before checkout, after confirmation, or as part of the final ticket?
- What should happen if payment mode is Others?

5. Activity 1 — Creation of Service Catalog Item

The primary UI entry point is a Service Catalog item named Book A Metro Ticket. It acts as the passenger booking form and hosts variables, UI policies and Catalog Client Scripts.

Procedure

- Open the Application Navigator.
- Click All and search for Service Catalog.
- Under Service Catalog, navigate to Maintain Items.
- Click New.
- Enter the item name as Book A Metro Ticket.
- Select Catalog → Service Catalog.
- Select the Services category.
- Enter a short description if required.
- Save the catalog item.

Suggested short description

“Book a metro journey by selecting source, destination, journey type, passenger count and payment details. Fare is calculated dynamically.”

6. Catalog Item Responsibilities

Component	Purpose
Catalog Item	Provides the user-facing booking entry point.
Variables	Capture journey and payment information.
Catalog Client Scripts	Perform dynamic client-side behavior such as fare calculation and submit handling.
Catalog UI Policies	Control visibility, mandatory state and behavior based on choices.
Widget	Provides a custom QR-ticket presentation when required.

7. UI Structure

Section	Fields
Journey Details	Starting From, Going To, Type of Journey, No. of Passengers
Fare Details	Amount for Single Journey, Amount Including Return
Payment	Mode of Payment, Enter Payment Mode
Ticket Output	QR-ticket presentation / confirmation experience



Reference screenshot: Service Catalog / Maintain Items navigation.

8. Activity 2 — Variables Configuration

Variables are the input controls displayed on the Book A Metro Ticket catalog form. Each variable should have a meaningful label, appropriate type, internal name and validation behavior.

Variable configuration procedure

- Open the catalog item.
- Open the Variables related list and click New.
- Select reference, multiple choice, select box or single-line text as appropriate.
- Enter the variable question/label and stable internal name for scripting.
- Configure choices or reference table where required.
- Add tooltip/example text when useful.
- Set mandatory/read-only behavior according to the requirement.
- Save and test the variable on the catalog form.

9. Variables to Create

S.No	Variable / Question	Type	Choices / Reference
1	Starting From	Reference	u_metro_station_detail(s)
2	Going To	Reference	u_metro_station_detail(s)
3	Type of Journey	Multiple Choice	Single Journey (1), Return Journey (2)
4	No. of Passengers	Select Box	1, 2, 3, 4
5	Amount for Single Journey	Single Line Text	Calculated fare
6	Amount Including Return	Single Line Text	Calculated return fare
7	Mode of Payment	Multiple Choice	UPI, Card, Others
8	Enter Payment Mode	Single Line Text	Shown when Others is selected

10. Variable Names Used by Scripts

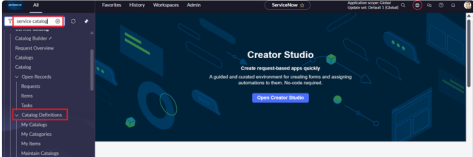
Display Label	Example Internal Name
Starting From	starting_from
Going To	going_to
Type of Journey	type_of_journey
No. of Passengers	no_of_passengers
Amount for Single Journey	amount_for_single_journey
Amount Including Return	amount_including_return
Mode of Payment	mode_of_payment
Enter Payment Mode	enter_payment_mode

Important: The exact variable names in the catalog item must match the names referenced by client scripts.

Description

Activity 1: Creation of Service Catalog

- Navigate to Application navigator
- Click on All >> search for Service Catalog
- Under Service Catalog>> Maintain items
- Click on New
- Fill the details >> Name– Book A Metro Ticket
- Select Catalog>> Service Catalog
- Select Category>> Services
- Fill the Short Description, if required
- Click on Save.



Reference screenshot: variable configuration and variable list.

11. Catalog Client Script — Dynamic Fare Calculation

A Catalog Client Script reacts to user input. Here, the onChange script on the passenger-count variable reads source, destination and journey type, then populates the appropriate fare field.

Configuration Procedure

- Open Service Catalog → Maintain Items.
- Open Book A Metro Ticket.
- Open the item's related lists.
- Select Catalog Client Script → New.
- Set script name and UI Type = All.
- Select Type = onChange.
- Set the variable to No. of Passengers.
- Enter the fare-calculation script and save.
- Test route, journey type and passenger-count combinations.

12. Fare Rules Implemented

Route	Single / Passenger	Return / Passenger
Ameerpet → Madhapur	₹30	₹60
Ameerpet → LB Nagar	₹50	₹100
Ameerpet → Kukatpally	₹30	₹60
Ameerpet → Jubilee Hills	₹20	₹40

Calculation rule: Total Fare = Fare Per Passenger × Number Of Passengers.

13. Client Script — Core Implementation

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') return;

    var source = g_form.getValue('starting_from');
    var destination = g_form.getValue('going_to');
    var numPassengers = Number(newValue);
    var journeyType = g_form.getValue('type_of_journey');

    if (source == 'SOURCE_ID' && destination == 'MADHAPUR_ID') {
        if (journeyType == '1') {
            g_form.setValue('amount_for_single_journey',
                numPassengers * 30);
            g_form.clearValue('amount_including_return');
        } else if (journeyType == '2') {
            g_form.clearValue('amount_for_single_journey');
            g_form.setValue('amount_including_return',
                numPassengers * 60);
        }
    }
    // Repeat the same pattern for approved routes.
    else {
        g_form.clearValue('amount_for_single_journey');
        g_form.clearValue('amount_including_return');
    }
}
```

Implementation note: Replace route placeholder IDs with the actual station sys_ids. The supplied source also contains one extra closing brace at the end; remove it before saving.

14. Catalog UI Policy — Conditional Fields

A Catalog UI Policy controls presentation or behavior based on conditions. Here it is used so that Enter Payment Mode is shown only when the user selects Others for Mode of Payment.

Scenario

Mode of Payment = Others → Enter Payment Mode becomes visible.

Procedure

- Open Catalog Items and select Book A Metro Ticket.
- Open the Catalog UI Policy related list.
- Click New.
- Set Applies to = Catalog Item.
- Select Book A Metro Ticket.
- Add condition: Mode of Payment is Others.
- Save the UI Policy.
- Open UI Policy Actions and click New.
- Select Enter Payment Mode.
- Set Visible = True.
- Update and test the catalog form.

15. Why Use a UI Policy?

- Improves usability by hiding irrelevant fields.
- Reduces user confusion and unnecessary input.
- Centralizes simple conditional UI behavior.
- Avoids custom JavaScript when standard UI Policy behavior is sufficient.

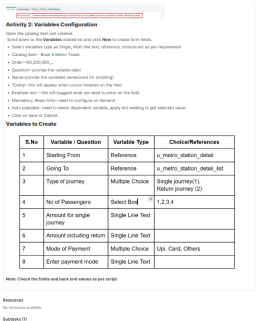
16. Catalog Client Script — QR Code Generation

A second Catalog Client Script is configured as an onSubmit script. It intercepts the first Checkout click, constructs a QR-code URL and opens the custom metro_qr_widget in a modal.

Checkout → onSubmit → Build Ticket URL → Build QR Image URL → Open spModal → metro_qr_widget → Continue checkout

Configuration

- Open Book A Metro Ticket → Related Lists → Catalog Client Script.
- Click New.
- Set it to apply to the catalog item.
- Select Type = onSubmit.
- Enter the QR modal logic and save.
- Test that the first submission opens the QR experience instead of immediately completing checkout.



Reference screenshot: Catalog Client Script / QR-related configuration.

17. QR Code Client Script

```
var metroQrDone = false;

function onSubmit() {
  var sysId = g_form.getUniqueValue();
  var dummyURL = 'https://example.com/ticket?id=' + sysId;

  var qrImageUrl =
    'https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=' +
    encodeURIComponent(dummyURL);

  spModal.open({
    title: "Your Metro QR Ticket – click Checkout once more to confirm",
    widget: "metro_qr_widget",
    widgetInput: { qr_url: qrImageUrl }
  }).then(function() {
    metroQrDone = true;
  });

  return false; // blocks the first Checkout click
}
```

Production note: The sample uses a placeholder ticket URL and an external QR image service. In production, the QR payload should point to a secure ticket-validation endpoint and should not expose sensitive information.

18. End-to-End UI/UX Flow

Step	User Action	System Behavior
1	Open Book A Metro Ticket	Display catalog form.
2	Select Starting From / Going To	Reference variables provide valid station choices.
3	Select Type of Journey	Single/Return option is captured.
4	Select No. of Passengers	onChange script evaluates route + journey type + count.
5	Review fare	Applicable fare is populated; non-applicable field is cleared.
6	Select payment mode	If Others is selected, Enter Payment Mode becomes visible.
7	Click Checkout	onSubmit opens QR widget and blocks first submission.
8	Confirm / Checkout again	User continues intended ticket submission flow.

19. Phase 3 Test Cases

Test	Scenario	Expected Result
TC01	Valid route + 1 passenger + Single	Correct single fare populated.
TC02	Valid route + 2 passengers + Return	Return fare = per-passenger return fare × 2.
TC03	Change passenger count	Fare recalculates dynamically.
TC04	Unsupported route	Fare fields are cleared.
TC05	Payment = Others	Enter Payment Mode becomes visible.
TC06	Payment = UPI/Card	Enter Payment Mode remains hidden.
TC07	First Checkout	QR modal opens and submission is blocked.
TC08	QR confirmation	User can continue checkout.
TC09	Missing journey input	Submission/processing is prevented per validation design.
TC10	Intended UI types	Catalog behavior is verified in the target interface(s).

20. Phase 3 Exit Criteria

- Catalog item is available and correctly categorized.
- All approved variables are created with correct types and internal names.
- Fare calculation works for every approved route/journey combination.
- UI Policy correctly controls Enter Payment Mode visibility.
- QR widget/modal behavior works as designed.
- Positive, negative and boundary test cases pass.
- Catalog experience is ready for deeper business logic, automation, notifications and production hardening.

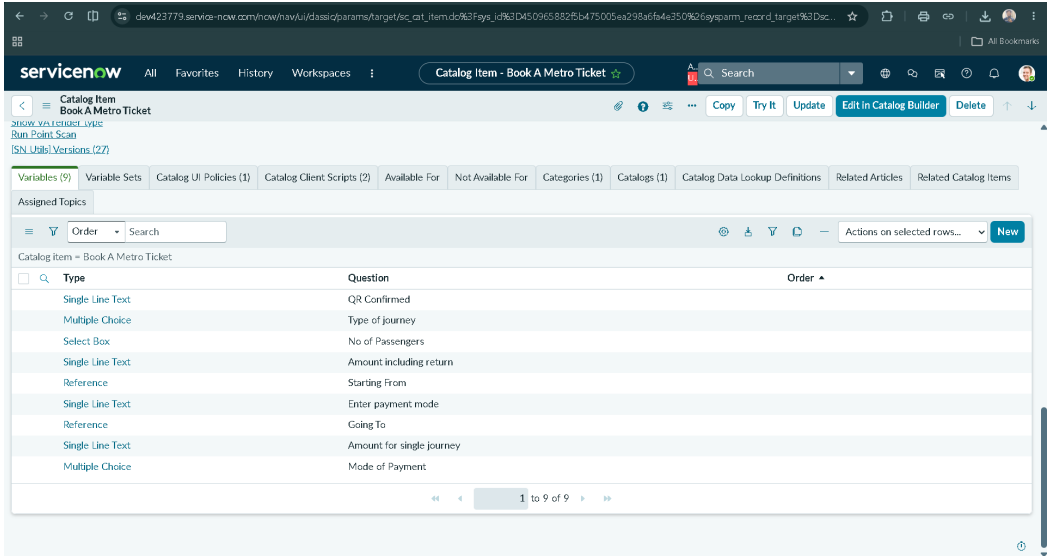
Final Outcome

Phase 3 transforms the Metro Ticket Generation System from a backend data model into a dynamic, user-friendly ServiceNow booking experience. The catalog item becomes the primary interface through which passengers provide journey details, receive calculated fare values and interact with the QR-ticket experience.

21. ServiceNow Implementation Evidence

The following screenshots are appended to the Phase 3 document as implementation evidence. They correspond directly to the catalog item, variable configuration, passenger-facing catalog form, and QR widget activities described in the Phase 3 material.

Figure 21. Catalog Item Variables — Book A Metro Ticket



ServiceNow Maintain Items view showing the eight configured variables: Starting From, Going To, Type of Journey, No Of Passengers, Amount for Single Journey, Amount Including Return, Mode of Payment and Enter Payment Mode.

Figure 22. Catalog Item Configuration — Book A Metro Ticket

The screenshot shows the ServiceNow 'Catalog Item - Book A Metro Ticket' configuration page. The browser address bar displays a URL starting with 'dev423773.servicenow.com'. The page header includes the ServiceNow logo and navigation tabs: 'All', 'Favorites', 'History', 'Workspaces', and 'Catalog Item - Book A Metro Ticket'. Below the header, a blue banner reads 'Build and modify items faster with the Improved Catalog Builder.' followed by instructions: 'Catalog items are goods or services available to order from the service catalog. Items can be anything from hardware, like tablets and phones, to software applications, to furniture and office supplies. • Enter a Name and Short description to display for the item. • Enter a Price, approvals, variables, and other information as needed.'

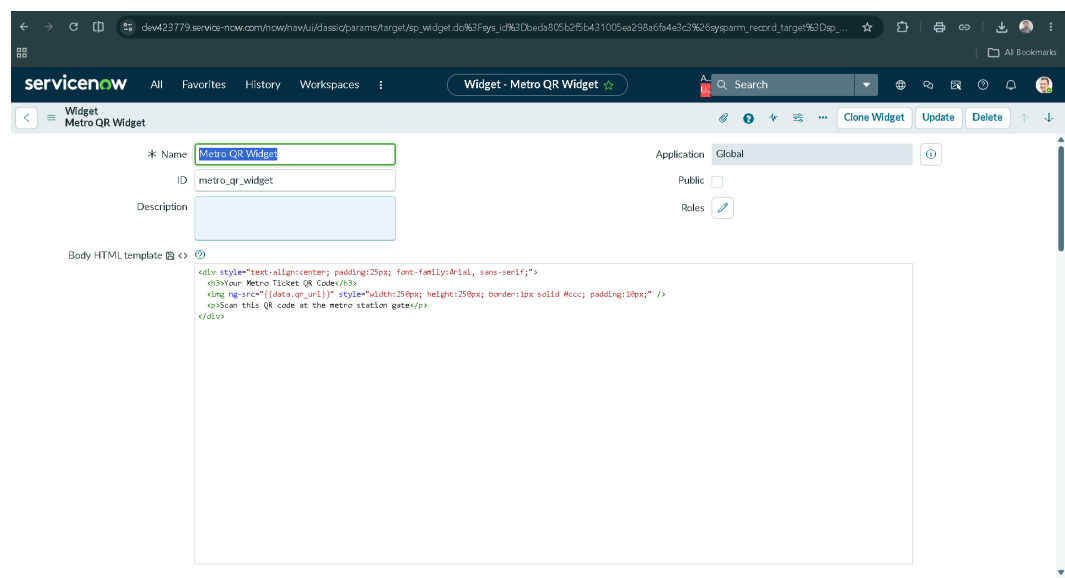
The configuration form contains the following fields and values:

- Name:** Book A Metro Ticket
- Application:** Global
- Catalogs:** Service Catalog
- Category:** Services
- Active:** ☒
- Fulfillment automation level:** Unspecified
- State:** -- None --
- Checked out:** -- None --
- Owner:** System Administrator
- Secondary owners:** (empty)

At the bottom, there are tabs for 'Item Details', 'Process Engine', 'Picture', 'Pricing', and 'Portal Settings'. The 'Item Details' tab is active, showing a 'Short description' field and a 'Description' field with a rich text editor toolbar below it.

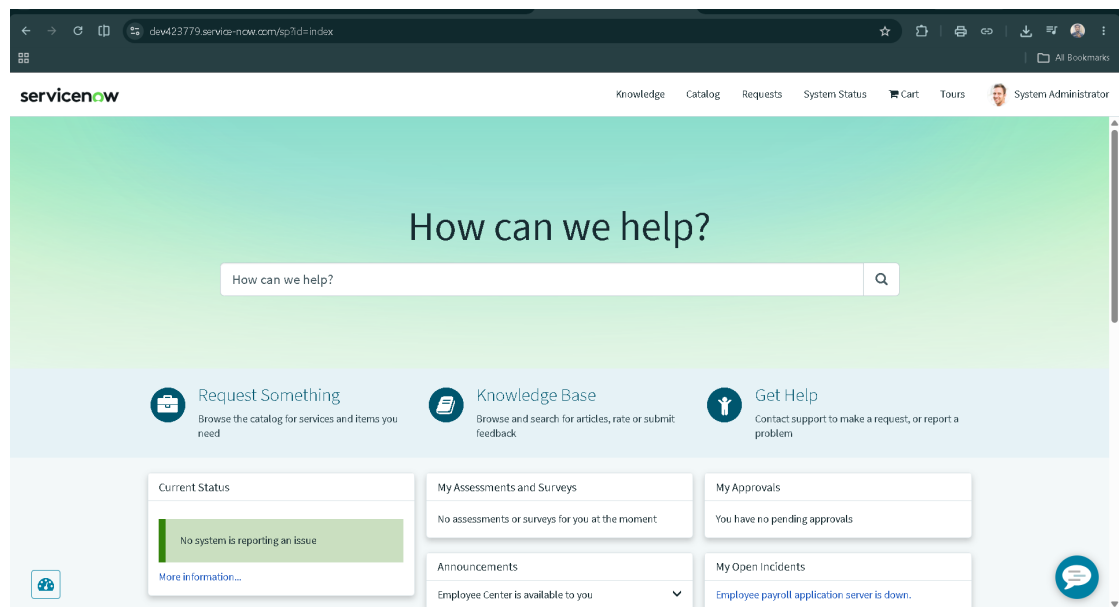
Catalog Item form showing the item name, Service Catalog, Services category, active state, application scope and the metro ticket short description.

Figure 23. Service Portal Home — Passenger Entry Point



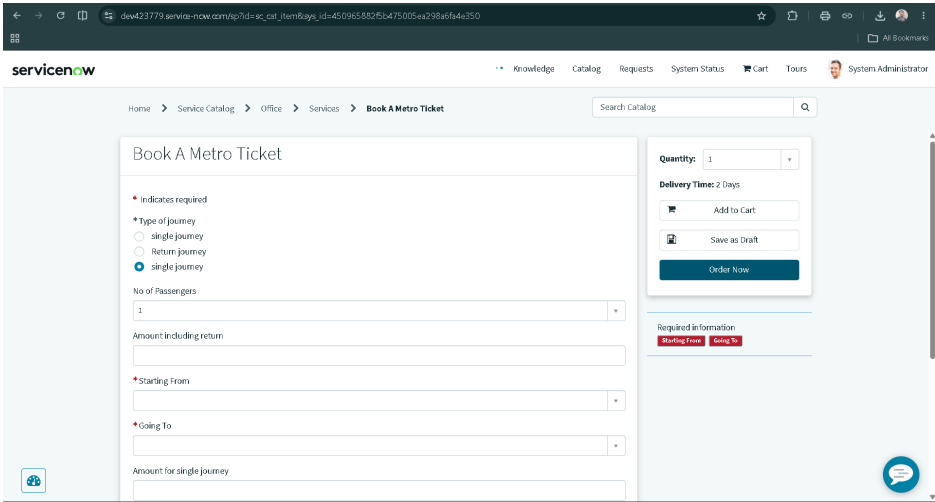
ServiceNow Service Portal landing page providing the passenger-facing interface and navigation to catalog services.

Figure 24. Book A Metro Ticket — Journey Details



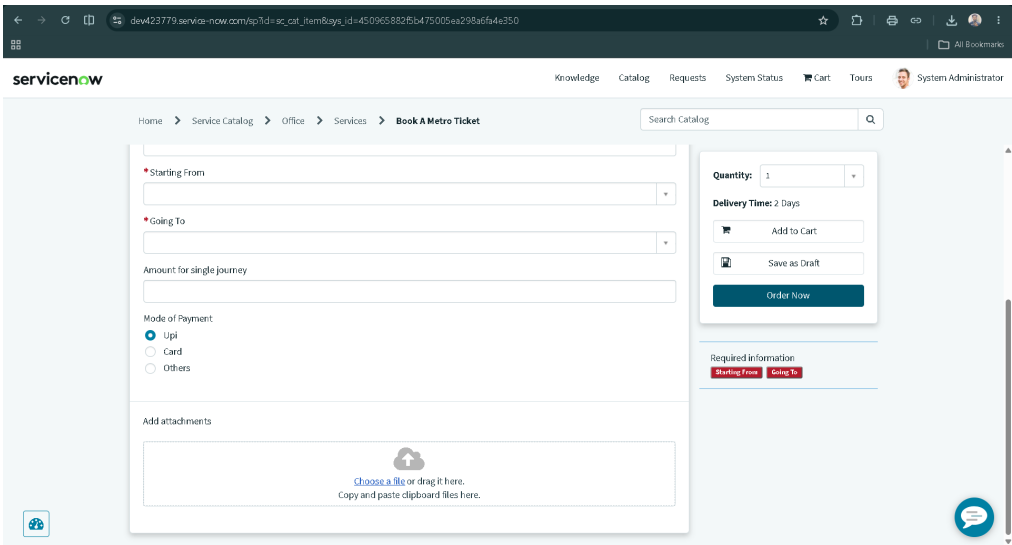
Passenger-facing catalog form showing required Starting From and Going To fields, Single/Return journey selection, passenger count and calculated fare fields.

Figure 25. Book A Metro Ticket — Fare and Payment Section



Lower portion of the catalog form showing passenger count, fare outputs, Enter Payment Mode and checkout controls.

Figure 26. Metro QR Widget Configuration



ServiceNow widget record for Metro QR Widget, including the widget ID and Body HTML template that displays the generated QR image and scan instruction.